



**SANTO
TOMÁS**[®]
INSTITUTO PROFESIONAL

ÁREA INFORMÁTICA

ASIGNATURA: Programación Web

INFORME: Git/GitHub.

Estudiantes: Luis Castro, Luis Liberona

Fecha: 19/05/2026

Docente: Rodrigo Gonzales

Introducción

GitHub

Hoy en día GitHub es una herramienta popular y muy utilizada en el área de informática, se podría decir que es como un 'LinkedIn' para programadores, GitHub es muy útil para trabajar de forma colaborativa en código, permite ver todos los cambios que se le han hecho al proyecto para ayudar con el control de versiones con los famosos 'repositorios'.

GitHub te deja hacer:

- Mostrar y/o compartir tu código.
- Rastrear y gestionar los cambios a tu código.
- Dejar que otros vean tu código y sugerir mejoras.
- Colaborar en un proyecto compartido gracias Git.

Git

Git es un control de versión que inteligentemente rastrea cualquier cambio que se haga en un archivo, Git es muy útil cuando múltiples personas trabajan en el mismo código, haciendo cambios continuos al mismo tiempo.

GitHub permite:

- Crear una nueva rama desde el 'main' que tu y tus colaboradores están trabajando.
- Hacer cambios independientemente y resguardada mente en tu propia rama persona.
- Permite inteligentemente unir los cambios específicos que hayas realizado en tu rama principal hacia el 'main'
- Deja de rastrear los cambios y actualizaciones que tu y tus colaboradores hayan hecho.

1. Glosario de conceptos importantes

- Repositorio:

Es un contenedor en donde se encuentra todo el trabajo del proyecto, además del historial de cambios que se han realizado.

- Control de versiones

Es una gestión de los diferentes tipos de versiones o iteraciones que existen en el proyecto, permitiendo saber que se cambio y quien lo cambio.

- Staging area

Es un espacio temporal en donde se colocan los siguientes cambios que quieres incluir en tu próximo guardado histórico.

- Commit history

Es la línea oficial del proyecto, muestra todos los cambios o 'Commits' que se hayan realizado.

- Conflicto

Situación que ocurre cuando varios desarrolladores modifican la misma línea en el mismo archivo en ramas distintas y luego tratan de unirlos.

2. Los 10 comandos más útiles y importantes de Git

1- git init:

Puede convertir cualquier carpeta común en un repositorio.

2- git clone [URL]:

Descarga una copia de un proyecto que ya existe en línea.

3- git status:

Ayuda a comprender el estado actual de tu copia de trabajo local.

4- git add [archivo]:

Pasa los archivos modificados a la 'Staging area'.

5- git commit:

Crea una 'snapshot' (una foto) de los cambios que hay en la Staging area y los guarda indefinidamente en el historial, se le puede añadir -m [mensaje] para añadir un comentario.

6- git branch [nombre]:

Crea una línea de tiempo paralela para poder experimentar o programar una nueva función sin alterar el código principal.

7- git checkout:

Permite saltar de una rama a otra para trabajar en diferentes partes del proyecto.

8- git merge [nombre de la rama]:

Fusiona el trabajo de una rama que tienes seleccionada con otra rama.

9- git push:

Sube todos los commits que tienes guardados localmente hacia un repositorio en la nube.

10- git pull:

Trae los últimos cambios que tus compañeros del proyecto han subido a la nube y los combina con tus proyectos locales en tu computadora.

3. Los 5 comandos más útiles de GitHub

-1 git clone [URL de GitHub]:

Permite descargar una copia de un proyecto en GitHub, además de descargar el historial del proyecto.

2- git remote add origin [URL de GitHub]:

Sirve para unir un proyecto local con un repositorio vacío creado en GitHub.

3- git push -u origin [nombre de rama]:

Sirve para guardar en la nube el proyecto elegido, para que luego sea visto por el resto del equipo.

4- git pull:

Permite descargar los cambios actualizados de un proyecto hacia tu computadora local y luego lo fusiona con el proyecto local.

5- git fetch:

Permite descargar la información de lo que hicieron los demás en el proyecto, a diferencia del pull este no afecta los archivos locales.

4. Alternativas a GitHub

- GitLab:

GitLab ofrece herramientas nativas y robustas para automatizar pruebas y despliegue de software, es más usado en el ámbito empresarial especialmente para DevOps.

- BitBucket:

Al igual que Gitlab, BitBucket se centra más en el ámbito empresarial con herramientas nativas además de ser una opción barata, si tienes muchos proyectos privados con pocos usuarios por proyecto.

- AzureDevOps:

Azure se centra más en el ámbito empresarial, este es propio de Microsoft, que además de tener repositorios Git, ofrece herramientas de planificación, automatización y despliegue a la nube.

- AWS CodeCommit:

El dueño de CodeCommit es Amazon por lo cual es la mejor opción para integrar los servicios de AWS.

Básicamente todo lo que tienen en común estas alternativas es que son especializadas ya sea para equipos pequeños, proyectos privados o para DevOps, uno tiene que pensar en el proyecto y elegir la plataforma que más se conforme con lo que el equipo busca.

Conclusiones

GitHub al igual que Git son herramientas sumamente importantes para el desarrollo de aplicaciones, programación, páginas web, etc, gracias a que facilita el trabajo colaborativo entre los miembros del equipo permitiendo así la mejor eficacia y desempeño en el desarrollo de la aplicación.

Git permite sincronizar todo el aporte de cada uno de los integrantes del equipo de manera intuitiva, luego GitHub se encarga de guardar todo el proyecto en la nube.

Al dominar GitHub y Git y saberse los comandos más importantes, uno se puede liberar un enorme carga de tener que sincronizar y ayuda a ordenar el flujo de trabajo para que esta sea más eficaz.